

1、电机概述

永磁同步电机主要由定子和转子两个部分组成，定子与电机壳体刚性连接，转子可以转动并粘贴有永磁体。

我们知道，在永磁体外部某个方向施加一个恒定方向的磁场，永磁体受到外部磁场会产生引力和斥力，产生旋转的力矩，最终永磁体被吸引、旋转、并静止在与外部施加磁场平行的方向。一个典型的例子就是在地球磁场下的指南针，指南针会转动到指向地球南北极的方向（磁针与地磁方向平行）。同样的，如果这个固定的磁场开始转动，那么永磁体也会跟着转动，永磁体会尽量跟随外部磁场与自身磁场平行夹角最小的方向转动。

这样我们就可以通过旋转磁场来令永磁体旋转到指定角度。同时，永磁体在外部施加的磁场中，所产生的力矩大小和永磁体与磁场方向的夹角有关，所以我們也可以通过控制外部磁场方向和永磁体磁场方向的夹角来控制永磁体产生的力矩。

回到电机的角度来说，就是我们可以通过控制定子上三个绕组电流的大小与电流方向，来控制定子的磁场方向与磁场大小，从而达到了控制转子的输出扭矩。而这种控制方法就是永磁同步电机的矢量控制（Field-Oriented Control，以下简称 FOC）。

2、FOC 控制简介

FOC 控制有许多独特的优势，它可以让我们对永磁同步电机进行“像素级”的控制，实现很多传统电机控制方法所无法达到的效果：

1. 可以在低转速下保持精确控制；
2. 可以很好地实现电机换向旋转；
3. 能够对电机进行力矩、速度、位置三个闭环控制；
4. FOC 控制的永磁同步电机噪音较小。

正如永磁同步电机概述节所述，FOC 控制的基础就是旋转磁场。在下图的磁场矢量中，我们看到电机定子的三个线圈 a、b、c 可以产生三个方向的磁场 B_a 、 B_b 、 B_c 它们能合成电机中的磁场 B 。FOC 控制器根据当前永磁体转子的角度、角速度、期望的输出力矩以及采样测量得到的 a、b、c 三个线圈的电流，计算得到三个线圈的电压通断状态与通断时间，进而通过 MOS 管来控制三个线圈的电压通断。这样就可以合成我们期望的磁场 B ，进而拖动电机转子按照我们期望的方式运动。

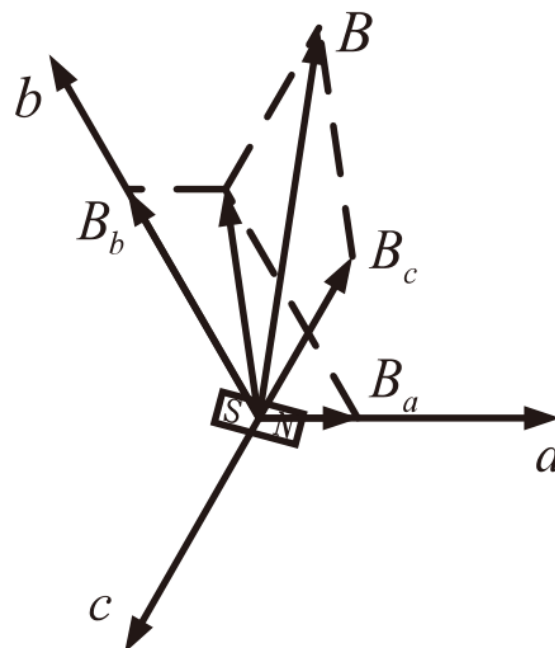


图 1 FOC 的磁场矢量

3、电机硬件与编码器概述

机器人关节电机的核心构件是电机驱动板、定子、转子和行星减速器。因为电机适合在高转速低力矩的工况下工作，而我们的机器人需要的是低转速高力矩，因此电机转子需要通过一个减速器减速之后，再输出力矩。

编码器（encoder）是用来测量旋转角度的传感器。编码器分为增量编码器、多圈绝对位置编码器与单圈绝对位置编码器等多种。我们在此只详细讨论关节电机实际应用的单圈绝对位置编码器。

关节电机的单圈绝对位置编码器安装在电机转子上。对于单圈绝对位置编码器（以下简称编码器），可以将它当做一个“时钟表盘”。我们每次看表的时候，都可以读到当前的日期和时间，例如 4 月 1 日 23 点。如果时间经过了 2 个小时，那么时钟转过了 4 月 1 日 24 点，日期会增加 1 天变成 4 月 2 日，时间重新从 0 点开始计时，变成了 4 月 2 日 1 点。为了方便计算经过的时间，我们也可以说现在是 4 月 1 日 25 点。看上去 25 点超过了一天 24 个小时的范围，实际上是因为日期增加了 1 天。

单圈绝对位置编码器也是同样的道理。每次开机上电后，我们的转子可能处于任意位置，编码器会告诉我们转子所处的角度位置（0 至 2π 之间的某个值）。如果转子旋转转过了 2π 这个角度位置，编码器也能够记录转过的圈数增加了 1，从而输出一个超出 0 至 2π 范围的角度位置。看上去编码器也能够输出超过一圈的角度位置，那么为什么叫它“单圈”绝对位置编码器呢？原因在于这种编码器在断电之后不能够储存之前旋转的圈数，我们以下面这个例子来说明。

假设当前编码器输出的角度值为 2.3π ，这意味着编码器在开机之后经过了 2π ，旋转圈数从 0 变为 1，同时编码器当前位于 0.3π 这个位置。此刻我们将编码器关机，不做任何旋转，再将编码器开机，那么此时编码器输出的角度值就会变为 0.3π 。因为关机之后旋转圈数重置为 0，所以编码器只会输出当前位置 0.3π 。

4、电机的混合控制

关节电机作为一个高度集成的动力单元，其内部已经封装了电机底层的控制算法。作为用户，只需要给关节电机发送相关的命令，电机就能完成从接收命令到关节力矩输出的全部工作。

对于电机的底层控制算法，唯一需要的控制目标就是输出力矩。可是对于机器人，我们通常需要给关节设定位置、速度和力矩。这时就需要对关节电机进行混合控制。

宇树科技的关节电机包含如下 5 个控制指令：

1. 前馈力矩： τ_{ff} ；
2. 期望角度位置： p_{des} ；
3. 期望角速度： ω_{des} ；
4. 位置刚度： k_p ；
5. 阻尼系数： k_d ；

在关节电机的混合控制中，使用 PD 控制器将电机在输出位置的偏差反馈到力矩输出上：

$$\tau = \tau_{ff} + k_p \times (p_{des} - p) + k_d \times (\omega_{des} - \omega)$$

式中， τ 为关节电机的电机转子输出力矩， p 为电机转子的当前角度位置， ω 为电机转子的角速度。在实际使用关节电机时，需要注意将电机输出端的控制目标量与发送的电机转子的指令进行换算。

5、电机的线路连接

我们使用单总线（1-Wire）作为物理层。物理层是指使用何种物理现象来表达和传输信息。实际上，可以将单总线视为一种基于单线通信的串行接口。

单总线仅利用一根数据线（以及公共地线）进行双向数据传输。该总线采用特殊的通信时序，在同一根线上实现设备数据传输。其单线连接方式极大简化了线缆连接，提高了硬件连接的可靠性，并降低了布线成本。

单总线信号采用和 TTL 相同的串口时序表示逻辑电平，由于单总线仅使用一根数据线进行双向通信，因此同一时刻总线上只能进行一个方向的通信，即半双工通信。总线通常由一个主机（上位机）控制通

信时序。主机发送的指令中包含目标设备地址信息，总线上所有设备均会接收该指令，但只有地址匹配的设备会响应并回复数据，从而完成一轮通信。

J288/S288 电机在单条总线上最多可支持 15 个电机（ID 范围为 0~14）。需特别注意，总线上严禁出现 ID 相同的电机，否则会导致整条总线通讯异常；此外，波特率不同的电机也不得连接在同一总线上。

为了将运动控制指令发送给电机，我们需要通过串口将指令下发。电机通过单总线接口与上位机通信，通信速率（波特率）固定为：6Mbps（8N1）。

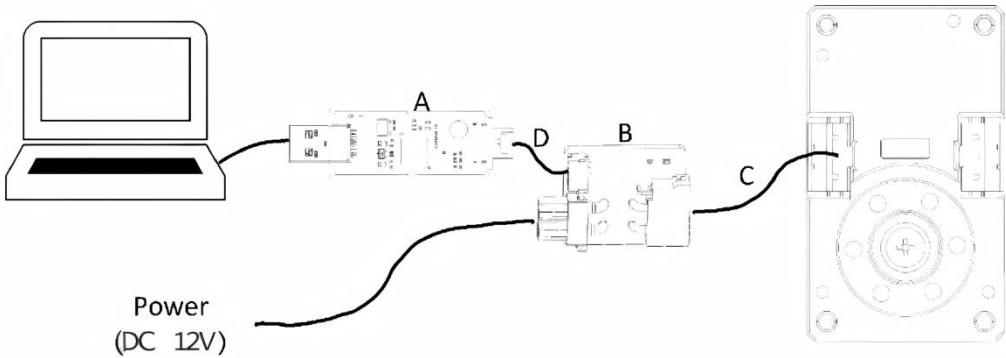


图 2 电机线路连接

如图 2 所示，接口采用 C 线缆连接至 B（XT30(2+2) 转接板），再通过 D（GH1.25-3 线缆）连接到 A（单总线转 USB 模块），最终连接到电脑。

在使用个人电脑控制电机时，需将单总线接口通过 USB 转单总线适配器连接至上位机。接通 12 V 直流电源后，电机绿色指示灯开始闪烁，表示电机已正常开机。

6、电机配置及快速使用

通过电机调试助手，您可以对电机执行以下配置操作：

- 查询及修改 ID
- 修改电机波特率
- 查询版本号
- 恢复电机模式
- 临时关闭引导
- 清除故障
- 电机复位
- 固件更新

通过电机调试助手，您可以快速使用电机，实现以下功能

- 电机运动控制
- 数据曲线绘制

具体操作方法详见《电机调试助手使用指南》。

7、电机协议

考虑到部分用户可能会使用特殊平台来控制电机，在此我们将讲解如何编写自定义的电机控制程序。根据本节介绍的方法，读者可以在任何满足硬件要求的平台上向电机发送控制命令并接收电机状态反馈。协议中的所有参数均为转子端数据，减速比为 288.35，若需将转子数据换算为输出端数据，则需乘以或

除以该减速比。

N215-288 电机采用串口通信，通信标准为单总线（1-Wire），波特率为 6.0 Mbps。串口的数据位为 8 bit，无奇偶校验位，停止位为 1 bit。需要注意的是为了提高电机的通信频率，我们使用了 6.0 Mbps 这一很高的波特率，用户需要检查自己的硬件是否支持这么高的波特率。如果您的硬件无法支持该波特率，可以使用 Unitree 的 USB 转单总线（1-Wire）模块。

在控制电机时，我们会通过串口给电机发送一个长度为 20 字节的命令，之后电机返回一个长度为 26 字节的信息。如果不给电机发送命令，那么电机也不会返回状态。电机发送命令格式与返回格式见下面结构体，结构体详细说明了各个字节的含义，下面我们解释其中部分细节。

首先，在发送给电机的命令中，第 5 和第 6 字节表示电机前馈力矩 τ_{ff} ，其中 256000 对应 1 牛·米 (NM)。这相当于我们对前馈力矩值乘以了 256000，然后将其赋值给一个 2 字节的带符号短整型 (signed short int) 变量。在此赋值过程中会进行强制取整，从而使我们能够仅用两个字节来传输前馈力矩 τ_{ff} 数据。当电机接收到此数据后，只需将其除以 256000，即可还原前馈力矩 τ_{ff} 的实际数值。这种方法虽然会损失一定精度，但对于实际应用场景而言完全满足需求。

此外，需要注意的是，对于一个 2 字节 (16 位) 的变量，其中 1 位用作符号位表示正负，因此只有 15 位可用于表示数值大小。这意味着命令中 τ_{ff} 的数值不能 2^{15} 超过特定上限。考虑到我们曾对原始数据 τ_{ff} 乘以了 256000，因此其绝对值存在相应的上限值：

$$|\tau_{ff}| < \frac{2^{15}}{256000} = 0.128$$

同时需要注意的是，由于赋值过程中存在强制取整操作， τ_{ff} 因此数值越大，保存的小数精度就越低。结构体变量注释中提到的“256000 表示 1”实际上是指上文所述的乘以 256000 的操作，其他变量中类似的“xx 表示 1”描述也遵循相同的原理。

并且，对于 2 个字节的 τ 变量来说，它的低位在前，即第 5 字节，高位在后，即第 6 字节。在发送命令和接收状态的末尾，我们可以看到一个 4 字节的 CRC32 校验。在命令发送之前，我们会计算这些命令字节的发送前 CRC 校验值，并且和命令一起发送给电机。当电机收到命令之后，还会根据收到的命令计算发送后 CRC 校验值。

如果数据传输过程中没有发生任何错误，那么发送前 CRC 校验值等于发送后 CRC 校验值。如果在数据传输过程中出现了数据错误，那么发送后 CRC 校验值的计算结果就与发送前 CRC 校验值不相等，电机就能够知道数据发生了损坏，从而帮助我们避免错误数据。

```
// 电机控制命令数据包 20 字节
typedef struct
{
    uint8_t head[2]; // 帧头 0xFE 0xEE 参与 CRC
    uint8_t id : 4; // 电机 ID: 0-14, 15 表示向所有电机广播数据(此时无返回)
    uint8_t status : 3; // 工作模式: 0. 锁定 1. FOC 闭环
    uint8_t timeout : 1; // Master->Motor: 0. 禁用超时保护 1. 开启 (默认 1s 超时)
    uint8_t res; // 保留位
    int16_t tor_des; // 期望转子输出扭矩 unit: N.m 256000 表示 1NM
    int16_t spd_des; // 期望转子输出速度 unit: rad/s 2.56/2π 表示 1rad/s
    int32_t pos_des; // 期望转子输出位置 unit: rad 32768/2π 表示 1rad
    int16_t k_pos; // 期望转子刚度系数 unit: N.m/rad 1280000 表示 1
    int16_t k_spd; // 期望转子阻尼系数 unit: N.m/(rad/s) 128000000 表示 1
    uint32_t CRC32; // CRC32
} ControlData_t;
```

```

// 电机控制模式
typedef struct
{
    uint8_t id : 4; // 电机 ID: 0-14, 15 表示向所有电机广播数据(此时无返回)
    uint8_t status : 3; // 工作模式: 0. 锁定 1. FOC 闭环
    uint8_t timeout : 1; // Master->Motor: 0. 禁用超时保护 1. 开启 (默认 1s 超时)
    // Motor->Master: 0. 没有超时 1. 触发超时保护(需要控制位发 0 清除)
} RIS_Mode_t;

// 2-电机返回数据 26 字节
typedef struct
{
    uint8_t NoUse[2]; // 通信中不存在(结构体 4 字节对齐), 在发送通信数据时必须去除该数据
    uint8_t head[2]; // 帧头 0xFC 0xEE 不参与 CRC
    RIS_Mode_t mode; // 电机控制模式
    int8_t temp; // 壳体温度: -128~127° C
    uint8_t sensor; // 绕组温度: 0-255° C
    uint8_t vol; // 电机端电压 0-127.5V 255 表示 127.5V
    int16_t torque; // 当前转子扭矩 256000 表示 1NM
    int16_t speed; // 当前转子速度 2.56/2π 表示 1rad/s
    int32_t pos; // 当前转子位置 32768/2π 表示 1rad
    uint32_t MError; // 电机错误码: 0x0. 正常 bit0 过流,...
    uint16_t OutPos : 13; // 输出端传感器 2^13 表示 1 圈
    uint16_t ExFlag : 3; // 警告码
    uint16_t res; // 保留位
    uint32_t CRC32; // CRC32
} MotorData_t;

// CRC32 校验
uint32_t crc32_core(uint32_t *ptr, uint32_t len)
{
    uint32_t xbit = 0;
    uint32_t data = 0;
    uint32_t CRC32 = 0xFFFFFFFF;
    const uint32_t dwPolynomial = 0x04c11db7;
    for (uint32_t i = 0; i < len; i++)
    {
        xbit = 0x80000000;
        data = ptr[i];
        for (uint32_t bits = 0; bits < 32; bits++)
        {
            if (CRC32 & 0x80000000)

```

```

{
    CRC32 <<= 1;
    CRC32 ^= dwPolynomial;
}
else
    CRC32 <<= 1;
if (data & xbit)
    CRC32 ^= dwPolynomial;

xbit >>= 1;
}
}
return CRC32;
}

```

8、电机故障码表

当电机发生故障时，会触发对应的错误状态标识，且多个错误状态可同时存在。故障排除后，错误状态将持续保留，直至接收到清除错误/复位指令或设备重新上电。

清除指令协议：电机控制协议中 status=6, tor_des=-256, spd_des=0, pos_des=0, k_pos=0, k_spd=0

复位指令协议：电机控制协议中 status=7

清除指令可在任意控制阶段插入，但当有错误正在发生或存在不支持该指令的错误时，该指令不会执行；执行后电机将保持在 Mode0（停机）状态，等待新指令。

复位指令需在电机停止时使用，用于控制电机重启并重新执行上电后的各项检测；收到该指令后执行时间约为 0.3 秒，在此期间电机不响应其他控制指令。

重新上电需在电机停止时操作，用于控制电机重启并重新执行上电后的各项检测；上电过程用时约为 1.3 秒，在此期间电机不响应其他控制指令。

故障码	位(bit)	说明	清除指令	复位指令/ 重新上电
0x01	0	过流		√
0x02	1	瞬态过压	√	√
0x04	2	持续过压	√	√
0x08	3	瞬态欠压	√	√
0x10	4	芯片过热	√	√
0x20	5	MOS 过热/冷	√	√
0x40	6	MOS 温度异常	√	√
0x100	8	壳体温度异常	√	√
0x200	9	绕组过热	√	√
0x400	10	转子编码器 1 错误		√
0x1000	12	输出编码器错误	√	√
0x2000	13	储存数据错误		√
0x4000	14	异常复位	√	√
0x10000	16	芯片验证错误		√

0x20000	17	标定模式		√
0x80000	19	驱动版本过低		√
0x100000	20	固件型号错误		√
0x400000	22	硬件过流		√

表 1.电机故障码表

9、警告码表

警告码	位(bit)	说明	清除指令	复位指令/ 重新上电
0x01	0	低压电源异常	√	√

表 2.电机警告码表